

实验六 图实验

基础练习一

【问题描述】

题目要求按图 10-7 建立无向连通图的邻接矩阵存储，并实现 InsertLine 和 DeleteLine 两个操作。本实验在 ex_1 中用 MatrixGraph 保存顶点数组 data 与二维数组 lines，顶点编号为 0 到 7。

程序需要能够输出原始邻接矩阵，向图中加入一条边，再删除同一条边，从而验证无向图矩阵的对称位置是否同步修改。

【基本思路】

邻接矩阵中 lines[i][j] 为 1 表示顶点 i 与顶点 j 之间有边，为 0 表示没有边。插入边时先用 find_pos 查找两个顶点在数组中的下标，然后同时设置 lines[i][j] 和 lines[j][i]；删除边时也同时清除这两个位置。

函数调用关系为：main() -> build_sample_graph() -> insert_line() / delete_line() / output()。

```
insert_line(G, a, b)
  i <- find_pos(a), j <- find_pos(b)
  lines[i][j] <- 1
  lines[j][i] <- 1
delete_line(G, a, b)
  若 lines[i][j] = 0 则返回 false
  lines[i][j] <- 0, lines[j][i] <- 0
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_6/ex_1/src/lib.rs，主程序位于对应 src/main.rs，测试代码位于 tests/test_fn.rs。

测试数据围绕题目图中的顶点、边和算法输出展开，重点检查插入删除、遍历序列、拓扑先后关系以及最小生成树总权值。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	图 10-7 原图	邻接矩阵中 0-1、0-5、1-2、1-4、2-3、3-4、5-6、5-7、6-7 对称位置均为 1。
2	插入边 2-4	lines[2][4] 与 lines[4][2] 同时变为 1。
3	删除边 2-4	两个矩阵位置重新变为 0，图恢复为原图。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是无向图矩阵的对称更新。当前实现用顺序查找顶点，写法直接，适合顶点数量较少的实验图；若以后顶点较多，可以增加顶点值到下标的映射表。

2.2 本实验设计的优点及可以改进的方向

把图的输入改为文件读取，并补充更多异常数据，例如不存在顶点、重复边和非连通带权图。

基础练习二

【问题描述】

题目要求按图 10-8 建立有向图的邻接表存储，并实现 InsertLine 和 DeleteLine 操作。本实验在 ex_2 中用 VexNode 表示顶点表，用 LineNode 表示每个顶点的出边链表。

图中顶点为 1、2、3、4、5、6，边包括 1->2、1->3、1->4、3->2、3->5、4->5、6->4、6->5。

【基本思路】

邻接表只记录出边，因此插入 1->2 只修改顶点 1 的边链表，不会自动生成 2->1。insert_line 使用头插法把新边接到链表前端；delete_line 从指定顶点的边链表中查找目标邻接点，找到后改变前驱指针完成删除。

函数调用关系为：main() -> build_sample_graph() -> insert_line() / delete_line() / output()。

```
insert_line(G, from, to)
    i <- find_pos(from), j <- find_pos(to)
    若 from 已有到 to 的边，则返回 false
    新建边结点 node.adjvex = j
```

```
node.next <- vxs[i].first
vxs[i].first <- node
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_6/ex_2/src/lib.rs, 主程序位于对应 src/main.rs, 测试代码位于 tests/test_fn.rs。

测试数据围绕题目图中的顶点、边和算法输出展开, 重点检查插入删除、遍历序列、拓扑先后关系以及最小生成树总权值。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	图 10-8 原图	邻接表输出 1: -> 4 -> 3 -> 2, 3: -> 5 -> 2, 6: -> 5 -> 4。
2	插入边 2->5	顶点 2 的边链表变为 2: -> 5。
3	删除边 2->5	顶点 2 的边链表恢复为空, 删除不存在边时返回 false。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是区分有向边的方向, 删除时只检查弧尾顶点的出边链表。头插法实现简单, 但会使输出顺序与插入顺序相反。

2.2 本实验设计的优点及可以改进的方向

把图的输入改为文件读取, 并补充更多异常数据, 例如不存在顶点、重复边和非连通带权图。

进阶练习一

【问题描述】

本题要求在基础练习一建立的邻接矩阵上, 从任意顶点开始做深度优先搜索, 并给出搜索序列。本实验在 ex_3 中复用 ex_1 的 MatrixGraph::dfs。

【基本思路】

DFS 使用 visited 数组记录顶点是否被访问。访问一个顶点后, 按矩阵列号从小到大检查邻接点, 遇到未访问顶点就递归深入。图 10-7 是连通图, 所以从 0 号顶点出发即可访问全部顶点。

函数调用关系为: `main() -> dfs_search(0) -> MatrixGraph::dfs(0) -> dfs_fun()`。

```
dfs(v)
    visited[v] <- true
    输出 v
    for j from 0 to vexnum-1
        if lines[v][j] = 1 且 visited[j] = false
            dfs(j)
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 `EXP/EXP_6/ex_3/src/lib.rs`, 主程序位于对应 `src/main.rs`, 测试代码位于 `tests/test_fn.rs`。

测试数据围绕题目图中的顶点、边和算法输出展开, 重点检查插入删除、遍历序列、拓扑先后关系以及最小生成树总权值。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	从顶点 0 开始	输出 [0, 1, 2, 3, 4, 5, 6, 7]。
2	自动测试	<code>dfs_from_matrix_visits_all_vexs</code> 通过, 说明 8 个顶点均被访问。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是递归访问时不能重复访问同一顶点。`visited` 数组既能防止回到已经访问过的顶点, 也能让搜索序列保持稳定。

进阶练习二

【问题描述】

本题要求在基础练习二建立的邻接表上，从任意顶点开始做广度优先搜索，并给出搜索序列。本实验在 ex_4 中复用 ex_2 的 ListGraph::bfs。

【基本思路】

BFS 使用数组模拟队列，顶点入队时立即标记 visited，出队时把顶点值加入结果序列。由于图 10-8 不是强连通图，从 1 出发遍历结束后，还要检查是否存在未访问顶点；若有，则继续从该顶点开始 BFS。

函数调用关系为：main() -> bfs_search(1) -> ListGraph::bfs(1) -> bfs_fun()。

```
bfs(start)
    start 入队并置 visited[start] = true
    while 队列非空
        v <- 出队
        输出 v
        将 v 的未访问邻接点依次入队
    扫描未访问顶点并继续遍历
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_6/ex_4/src/lib.rs，主程序位于对应 src/main.rs，测试代码位于 tests/test_fn.rs。

测试数据围绕题目图中的顶点、边和算法输出展开，重点检查插入删除、遍历序列、拓扑先后关系以及最小生成树总权值。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	从顶点 1 开始	输出 [1, 4, 3, 2, 5, 6]。
2	非强连通处理	访问 1 所在部分后，程序继续访问未到达的顶点 6。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是队列顺序和非强连通图的补充遍历。当前实现使用 Vec 与 head 下标模拟队列，避免频繁删除队头元素，逻辑比较清楚。

进阶练习三

【问题描述】

本题要求把基础练习一中建立的邻接矩阵转换为邻接表。本实验在 ex_5 中调用 MatrixGraph::to_list_graph，并输出转换后的表结构。

【基本思路】

转换时逐行扫描矩阵，若 lines[i][j] 为 1，就把 j 作为 i 的一个邻接点插入邻接表。由于插入函数采用头插法，扫描列时从后向前处理，这样最终输出顺序仍接近矩阵从小到大的列顺序。

函数调用关系为：main() -> matrix_to_list_text() -> build_sample_graph() -> to_list_graph() -> output()。

```
to_list_graph(G)
    新建空邻接表 L
    for i from 0 to vexnum-1
        for j from vexnum-1 down to 0
            if lines[i][j] = 1
                在 L 的第 i 个顶点链表中插入 j
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_6/ex_5/src/lib.rs，主程序位于对应 src/main.rs，测试代码位于 tests/test_fn.rs。

测试数据围绕题目图中的顶点、边和算法输出展开，重点检查插入删除、遍历序列、拓扑先后关系以及最小生成树总权值。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	顶点 0	转换后为 0: -> 1 -> 5。
2	顶点 7	转换后为 7: -> 5 -> 6。
3	全部顶点	8 个顶点均输出对应邻接表，没有丢失边。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是矩阵和邻接表之间的下标含义一致。实现中复用原图顶点数组，避免转换后顶点编号错位。

扩展练习一

【问题描述】

本题要求对图 10-8 所示有向图进行拓扑排序，且使用邻接表方式存储。本实验在 ex_6 中复用基础练习二的邻接表，并实现 topo_sort。

【基本思路】

拓扑排序先统计所有顶点入度，把入度为 0 的顶点压入栈。每次取出一个入度为 0 的顶点加入结果序列，同时删除它发出的边，使相邻顶点入度减 1；若某个相邻顶点入度变为 0，则继续入栈。

函数调用关系为：main() -> topo_sort_result() -> ListGraph::topo_sort()。

```
topo_sort(G)
    统计 indegree[]
    将所有入度为 0 的顶点入栈
    while 栈非空
        v <- 出栈并输出
        for v 的每个邻接点 w
            indegree[w] <- indegree[w] - 1
            若 indegree[w] = 0, 则 w 入栈
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_6/ex_6/src/lib.rs，主程序位于对应 src/main.rs，测试代码位于 tests/test_fn.rs。

测试数据围绕题目图中的顶点、边和算法输出展开，重点检查插入删除、遍历序列、拓扑先后关系以及最小生成树总权值。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	图 10-8	输出一种拓扑序列 [6, 1, 3, 2, 4, 5]。
2	边关系检查	1 位于 2 前, 3 位于 5 前, 6 位于 4 前, 符合有向边先后关系。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是入度数组的维护。由于栈中顶点的压入顺序会影响最终结果, 拓扑序列不唯一; 当前输出是一种合法序列。

附加题一

【问题描述】

本题要求在图的邻接矩阵存储结构基础上实现最小生成树 Prim 算法。本实验在 ex_7 中复用 ex_1 的 WeightedMatrixGraph, 边权来自当前代码中的 build_weighted_sample_graph。

【基本思路】

带权邻接矩阵用 INF 表示两个顶点之间无直接边, 主对角线为 0。Prim 从指定顶点开始, 把它加入生成树集合; lowcost[i] 保存当前生成树连接顶点 i 的最小边权, adjvex[i] 保存该边的另一个端点。每轮选出 lowcost 最小且未访问的顶点加入生成树, 再更新其它顶点的 lowcost。

函数调用关系为: main() -> prim_result(0) -> WeightedMatrixGraph::prim(0) -> total_weight()。

```
prim(start)
    visited[start] <- true
    初始化 lowcost 和 adjvex
    重复 vexnum-1 次
        选择未访问且 lowcost 最小的顶点 k
        输出 adjvex[k] 到 k 的边
        用 k 更新其它顶点的 lowcost
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_6/ex_7/src/lib.rs，主程序位于对应 src/main.rs，测试代码位于 tests/test_fn.rs。

测试数据围绕题目图中的顶点、边和算法输出展开，重点检查插入删除、遍历序列、拓扑先后关系以及最小生成树总权值。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	从顶点 0 开始	选出 7 条边：0-5、0-1、1-4、4-3、1-2、5-6、6-7。
2	总权值	总权值为 25，测试 prim_result_gets_min_tree 通过。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是 lowcost 与 adjvex 两个数组的同步更新。当前图共有 8 个顶点，生成树边数为 7，符合最小生成树的基本性质。

附加题二

【问题描述】

本题要求在边集数组存储结构基础上实现最小生成树 Kruskal 算法。本实验在 ex_8 中复用 ex_2 的 EdgeGraph，边集数组中每个 Edge 保存 from、to 和 weight。

【基本思路】

Krusal 算法先按边权从小到大排序，再逐条检查边的两个端点是否属于同一个集合。若两个端点不在同一集合，则加入该边，并合并集合；若已在同一集合，则跳过这条边以避免形成回路。

函数调用关系为：main() -> kruskal_result() -> EdgeGraph::kruskal() -> total_weight()。

```
kruskal(G)
  按 weight 从小到大排序 edges
  parent[i] <- i
  for e in edges
    m <- get_end(e.from), n <- get_end(e.to)
    if m != n
      选中 e, 并令 parent[m] = n
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 EXP/EXP_6/ex_8/src/lib.rs, 主程序位于对应 src/main.rs, 测试代码位于 tests/test_fn.rs。

测试数据围绕题目图中的顶点、边和算法输出展开, 重点检查插入删除、遍历序列、拓扑先后关系以及最小生成树总权值。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	边集数组	按权值依次选出 6-7、1-4、3-4、0-5、0-1、1-2、5-6。
2	总权值	总权值为 25, 测试 kruskal_result_gets_min_tree 通过。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是判断加入边后是否成环。